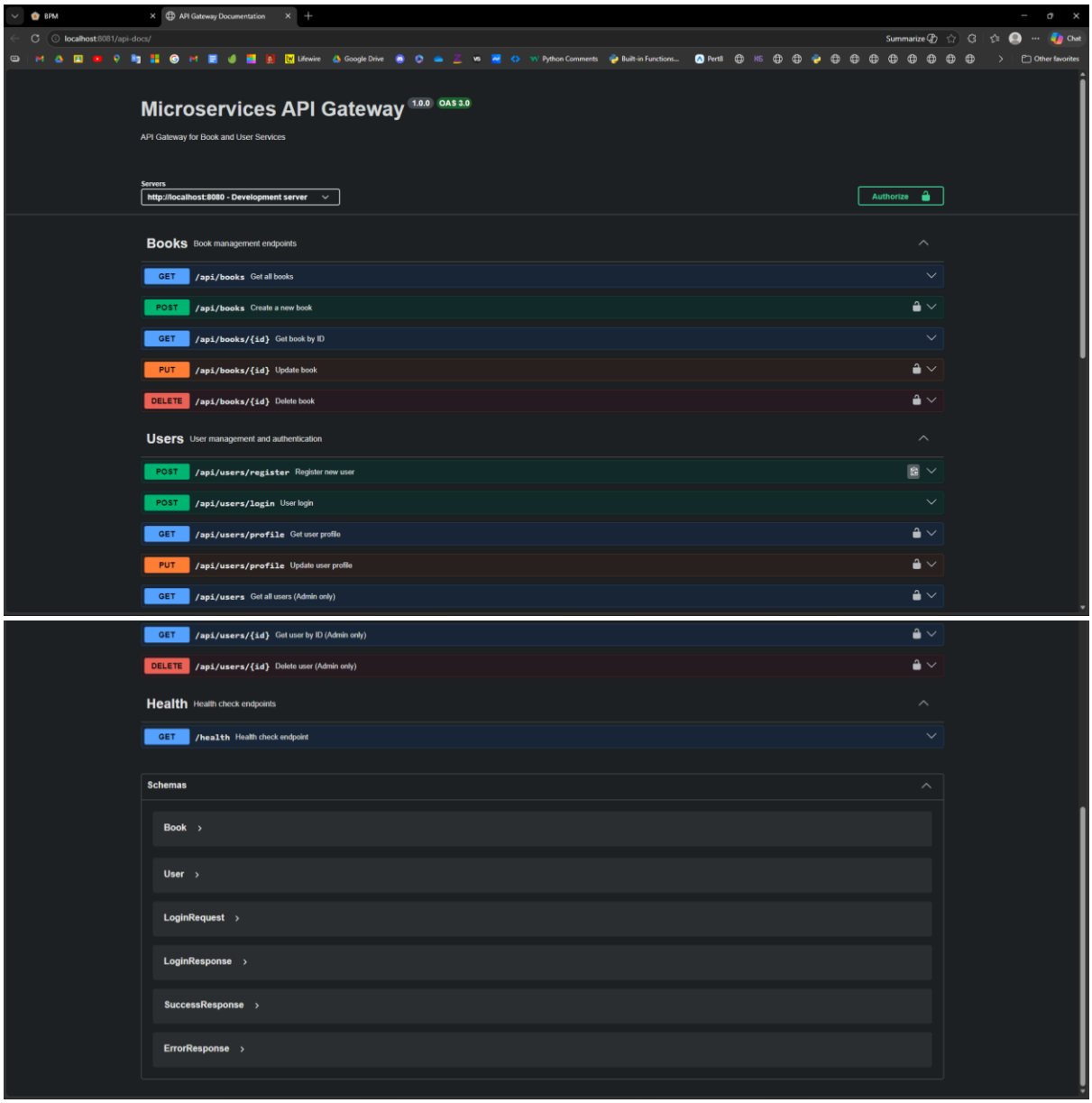
Nama: Joshua Taftfarel Petra Chrisensa

NPM: 232310002

Kelas: TI-23-KA




1. /health
   Pada endpoint /health dengan port 8080, ketika http mengirimkan hit request, server berhasil memberikan respon 200 → respon body berisi json
2. /api/books
   Pada endpoint dari books di swagger server berhasil megirimkan respon.
3. /api/users
   Pada endpoint users dibagian admin only diminta untuk memberikan JWT sebagai akses account

my-api-gateway > config > system.config.yml > {} db > {} redis

```yaml
# Core
db:
  redis:
    emulate: true
    namespace: EG
  Ctrl+L for Agent
#plugins:
  # express-gateway-plugin-example:
  #   param1: 'param from system.config'

crypto:
  cipherKey: sensitiveKey
  algorithm: aes256
  saltRounds: 10

# OAuth2 Settings
session:
  secret: keyboard cat
  resave: false
  saveUninitialized: false
accessTokens:
  timeToExpiry: 7200000
refreshTokens:
  timeToExpiry: 7200000
authorizationCodes:
  timeToExpiry: 300000
```

my-api-gateway > JS server.js > ...

```javascript
const path = require("path");
const express = require("express");
const gateway = require("express-gateway");

const swaggerUi = require("swagger-ui-express");
const swaggerSpec = require("./swagger");
console.log("🚀 Starting My API Gateway...");
console.log(`📁 Config directory: ${path.join(__dirname, "config")}`);
// Create Express app for Swagger
const swaggerApp = express();
swaggerApp.use(
  "/api-docs",
  swaggerUi.serve,
  swaggerUi.setup(swaggerSpec, {
    explorer: true,
    customCss: ".swagger-ui .topbar { display: none }",
    customSiteTitle: "API Gateway Documentation",
    customFavIcon: "/assets/favicon.ico",
  }),
);
// Start Swagger server on different port
const SWAGGER_PORT = 8081;
swaggerApp.listen(SWAGGER_PORT, () => {});
console.log(`
  📚
  Swagger UI running on: http://localhost:${SWAGGER_PORT}/api-docs`);
// Start Express Gateway
gateway()
  .load(path.join(__dirname, "config"))
  .run()
  .then(() => {
    console.log("\n" + "=".repeat(70));
    console.log("MY API GATEWAY STARTED SUCCESSFULLY");
    console.log("=".repeat(70));
    console.log(`Gateway URL: http://localhost:8080`);
    console.log(`Admin API: http://localhost:9876`);
    console.log(`Swagger UI: http://localhost:${SWAGGER_PORT}/api-docs`);
    console.log("\nConnected Services:");
    console.log(`Book Service: http://localhost:3001`);
    console.log(`User Service: http://localhost:3002`);
    console.log("\nAvailable Endpoints:");
    console.log(`   - Books API: http://localhost:8080/api/books`);
    console.log(`   - Users API: http://localhost:8080/api/users`);
    console.log(`   - Health Check: http://localhost:8080/health`);
    console.log("=".repeat(70) + "\n");
  })
  .catch((err) => {
    console.error("\nGATEWAY FAILED TO START");
    console.error("Error:", err.message);
    console.error("Stack:", err.stack);
    process.exit(1);
  });
// Graceful shutdown
process.on("SIGTERM", () => {
  console.log("\nShutting down gracefully...");
  process.exit(0);
});
process.on("SIGINT", () => {
  console.log("\nReceived SIGINT, shutting down gracefully...");
  process.exit(0);
});
```

my-api-gateway > config > gateway.config.yml > {} pipelines > {} swaggerPipeline > [ ] policies > {} 1 > [ ] log > {} 0 > {} action > [

```yaml
# HTTP listener configuration for the API Gateway
http:
  port: 8080

# Admin API configuration for managing the Express Gateway instance
admin:
  port: 9876
  host: localhost

# Define exposed API endpoints that the gateway will handle and listen for
apiEndpoints:
  # Endpoint grouping for book-related requests
  books:
    host: '*'
    paths: ['/api/books', '/api/books/*']
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE']

  # Endpoint grouping for user-related requests
  users:
    host: '*'
    paths: ['/api/users', '/api/users/*']
    methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE']

  # Internal health check endpoint
  health:
    host: '*'
    paths: '/health'
    methods: ['GET']

  # Swagger API documentation endpoint access
  swagger:
    host: '*'
    paths: '/api-docs*'
    methods: ['GET']

# Define backend microservices that the gateway will forward requests to
serviceEndpoints:
  # The upstream user URL for the Book service
  bookService:
    url: 'http://localhost:3001'

  # The upstream user URL for the User service
  userService:
    url: 'http://localhost:3002'

# List of enabled policies/plugins in Express Gateway that can be used in pipelines
policies:
  - basic-auth
  - cors
  - expression
  - key-auth
  - log
  - oauth2
  - proxy
  - rate-limit
  - request-transformer
  - response-transformer
```

my-api-gateway > config > gateway.config.yml > {} pipelines > {} swaggerPipeline > [ ] policies > {} 1 > [ ] log > {} 0 > {} action > message

```yaml
# Define pipelines that connect the incoming apiEndpoints to the backend serviceEndpoints
pipelines:
  # Pipeline to handle and process books requests
  bookPipeline:
    apiEndpoints:
      - books
    policies:
      # Enable Cross-Origin Resource Sharing (CORS) for external web clients
      - cors:
          - action:
              origin: '*'
              credentials: true
              methods: 'GET,HEAD,PUT,PATCH,POST,DELETE'
              allowedHeaders: 'Content-Type,Authorization'
      # Log every incoming request details
      - log:
          - action:
              message: '📚 Book Service Request: ${req.method} ${req.url}'
      # Add custom headers to the request before forwarding it to the backend service
      - request-transformer:
          - action:
              headers:
                add:
                  X-Gateway: "'my-api-gateway'"
                  X-Service: "'book-service'"
      # Proxy and forward the modified request to the upstream book service
      - proxy:
          - action:
              serviceEndpoint: bookService
              changeOrigin: true

  # Pipeline to handle and process users requests
  userPipeline:
    apiEndpoints:
      - users
    policies:
      # Enable CORS for user APIs
      - cors:
          - action:
              origin: '*'
              credentials: true
              methods: 'GET,HEAD,PUT,PATCH,POST,DELETE'
              allowedHeaders: 'Content-Type,Authorization'
      # Log every incoming request details
      - log:
          - action:
              message: '👤 User Service Request: ${req.method} ${req.url}'
      # Add custom headers for the user service backend
      - request-transformer:
          - action:
              headers:
                add:
                  X-Gateway: "'my-api-gateway'"
                  X-Service: "'user-service'"
      # Proxy the request to the upstream user service
      - proxy:
          - action:
              serviceEndpoint: userService
              changeOrigin: true
```

```yaml
# Pipeline to handle health check requests internally without forwarding
healthPipeline:
  apiEndpoints:
    - health
  policies:
    - cors:
    - log:
        - action:
            message: '💗 Health check'
    # Return a custom JSON response for health checks using inline JavaScript/Express code
    - expression:
        - action:
            jscode: |
              res.json({
                status: 'healthy',
                timestamp: new Date().toISOString(),
                gateway: 'my-api-gateway',
                services: {
                  books: 'http://localhost:3001',
                  users: 'http://localhost:3002'
                },
                documentation: 'http://localhost:8080/api-docs'
              });

# Pipeline to track Swagger documentation access
swaggerPipeline:
  apiEndpoints:
    - swagger
  policies:
    - cors:
        - action:
            origin: '*'
            credentials: true
    - log:
        - action:
            message: '📖 Swagger Documentation Access'
```